Object Oriented Programming

Mihai Dascălu

Exceptions

Scenario

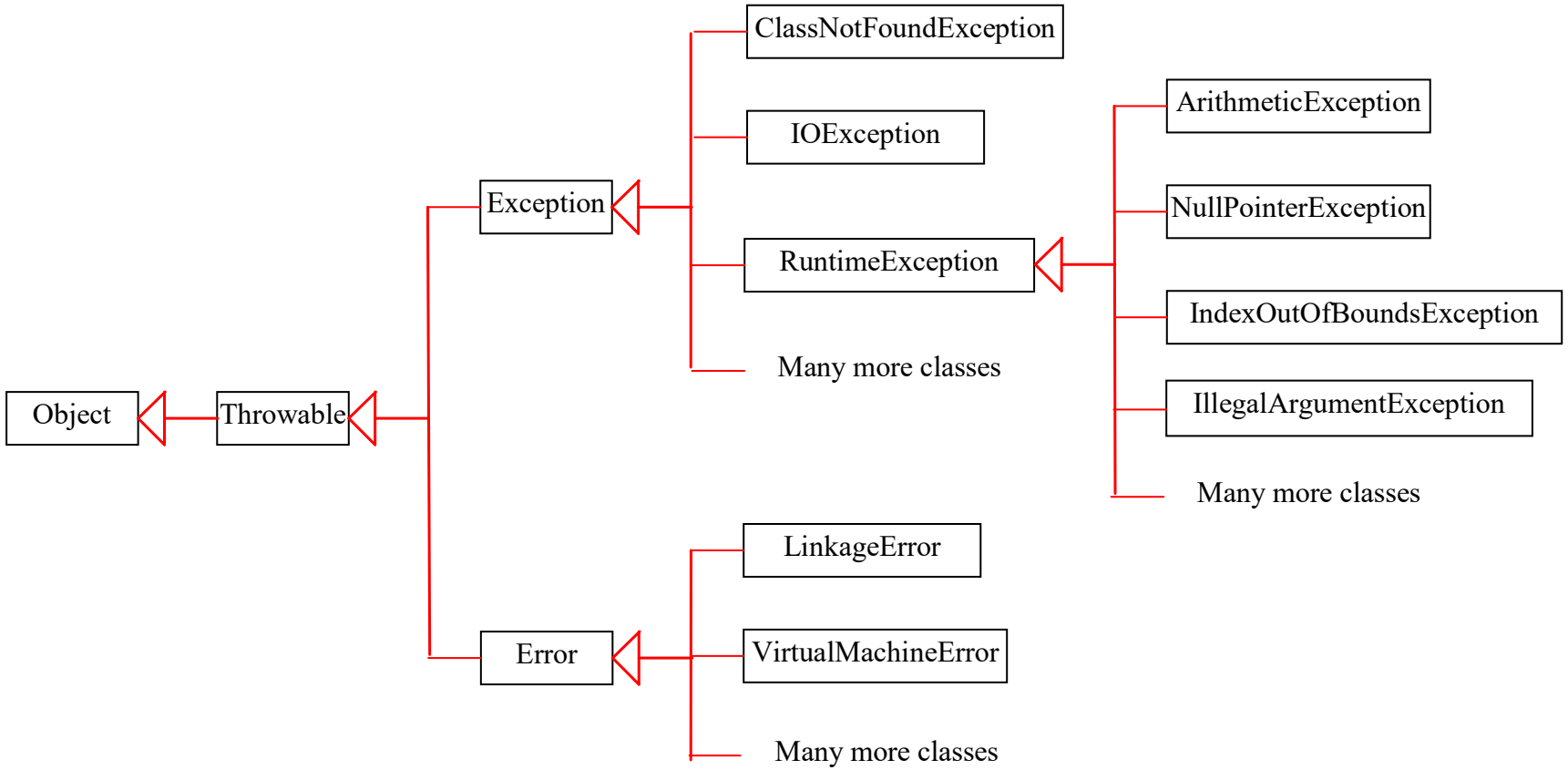
- When a program runs into a runtime error, the program terminates abnormally
- How can you handle runtime errors so that the program can continue to run or terminate gracefully?

Exception-Handling Overview

- Show runtime error
- Fix it using an if statement
- What if the runtime error occurs in a called method?
- Advantages of using exception handling -> enable a method to throw an exception to its caller
- Without this capability, a method must handle the exception or terminate the program.

Quotient

Exception Types

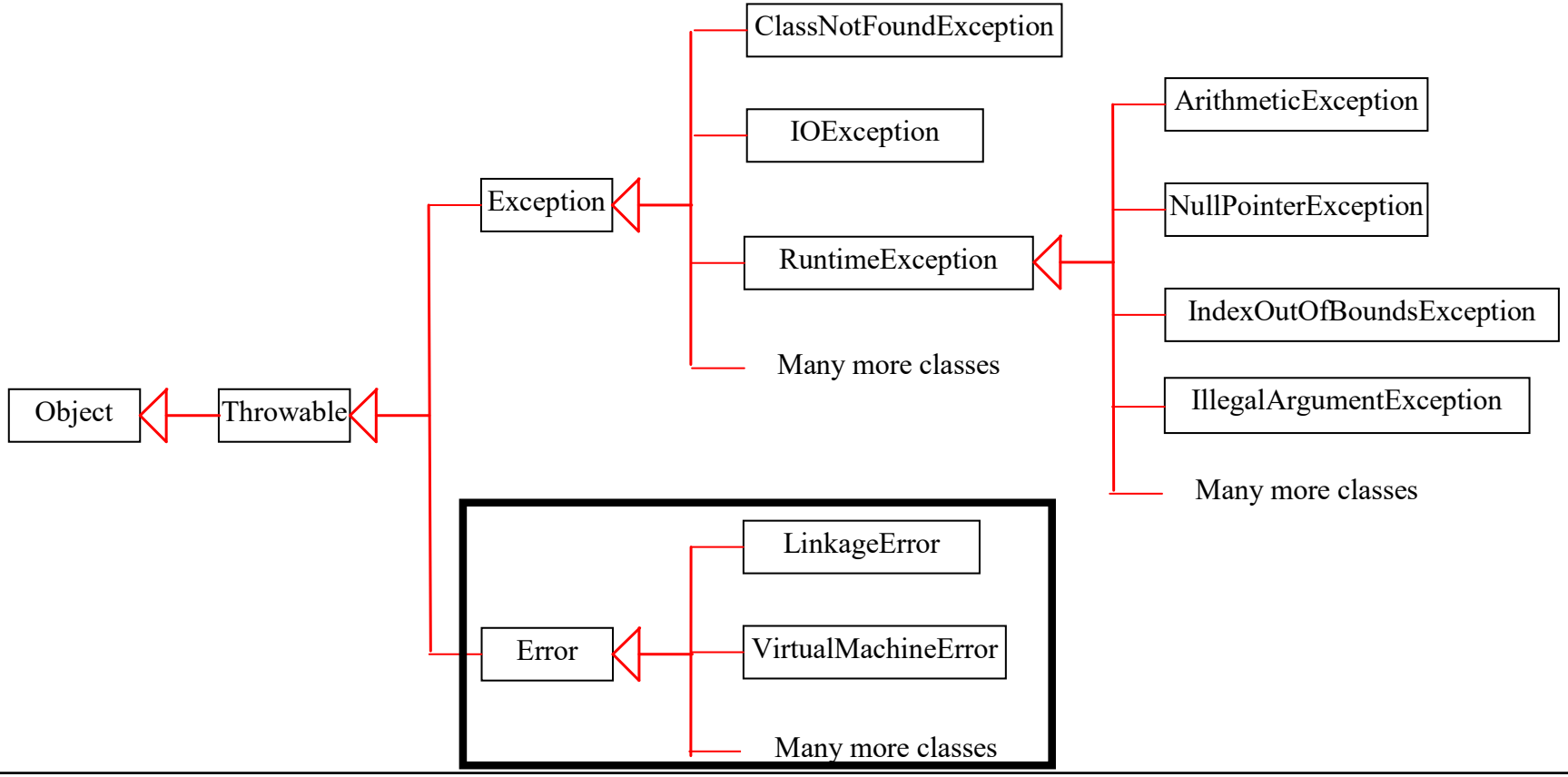


System Errors

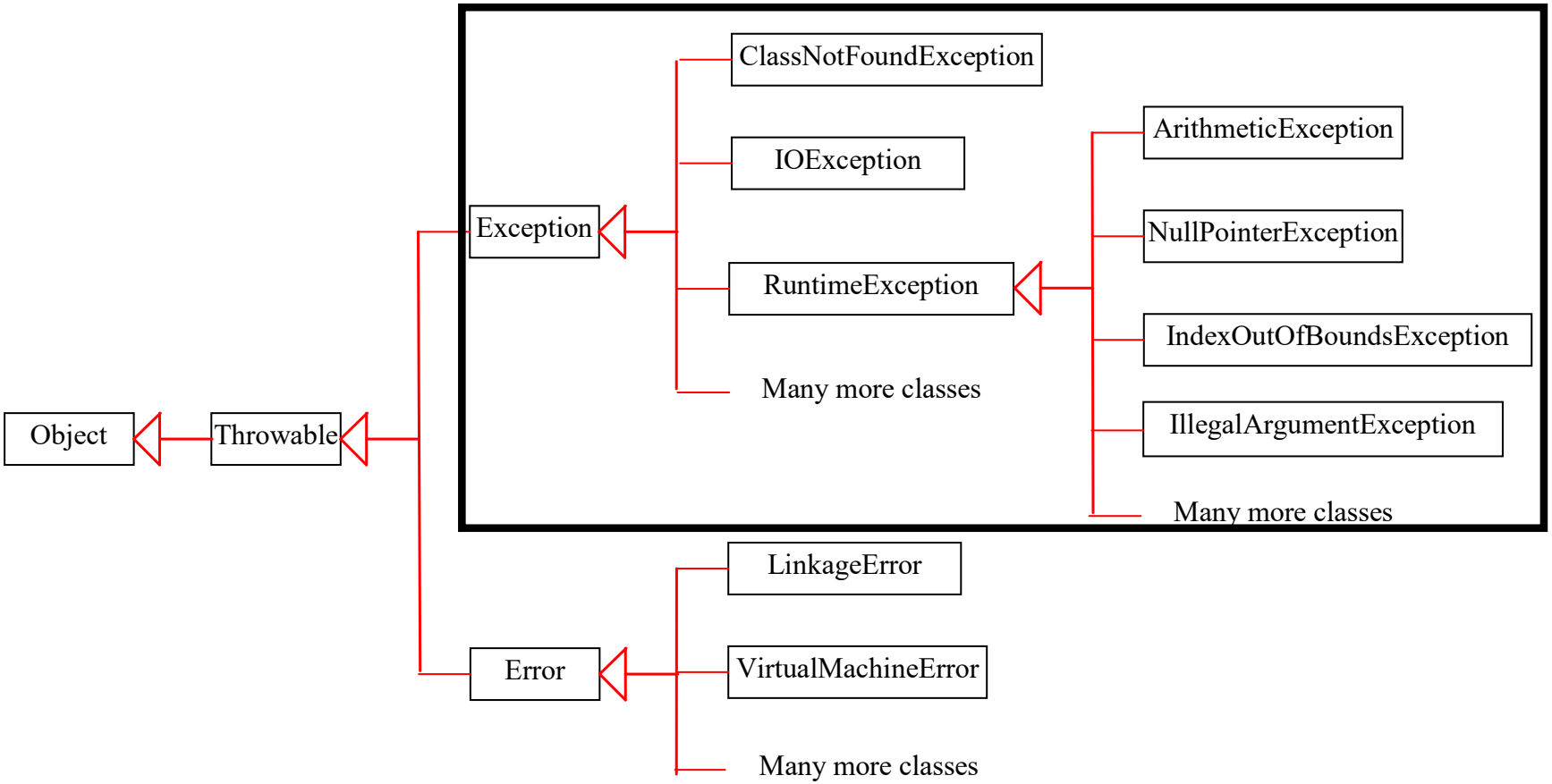
System errors are thrown by JVM and represented in the Error class

The Error class describes internal system errors & such errors rarely occur

If an error occurs, there is little you can do beyond notifying the user and trying to terminate the program gracefully



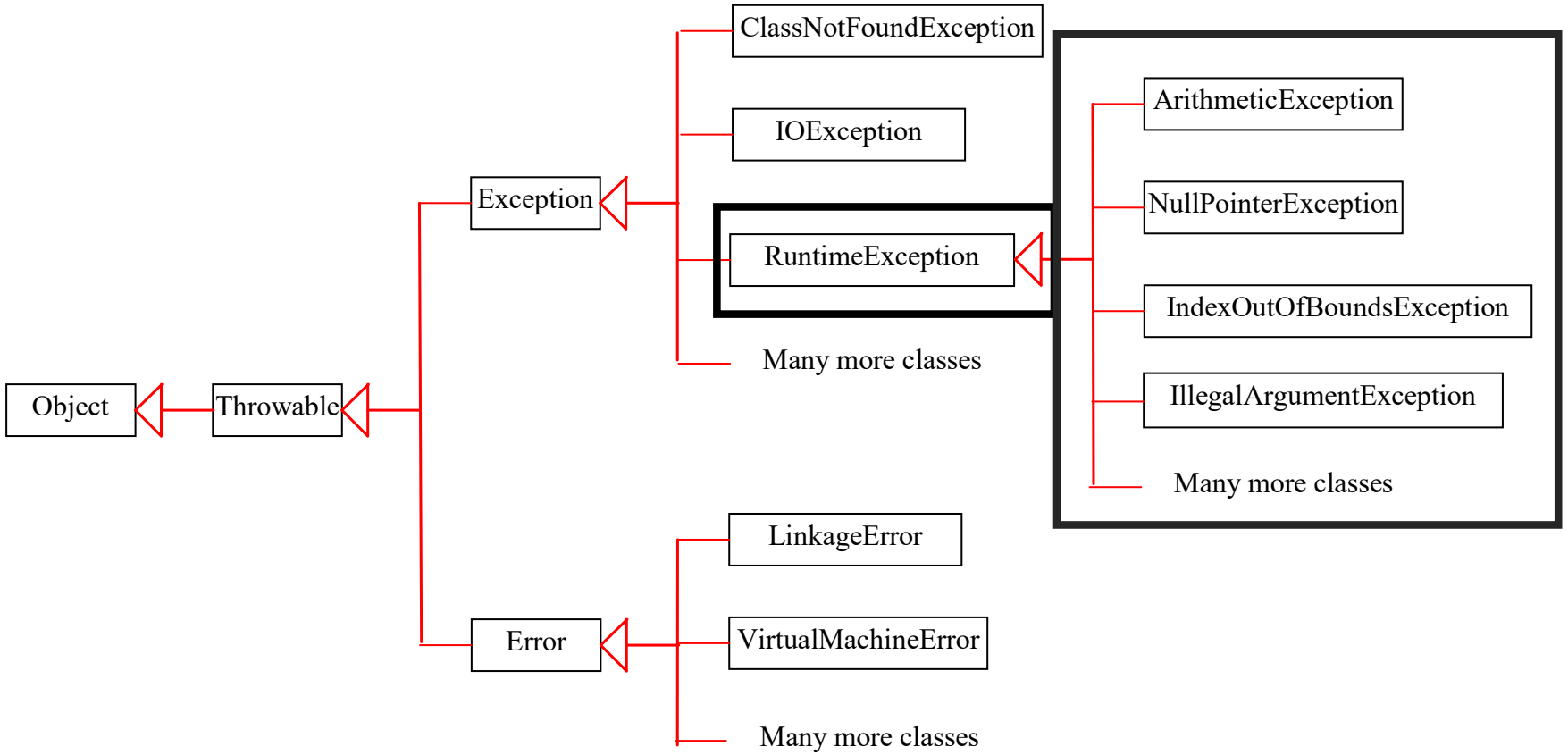
Exceptions



Exception describes errors caused by your program and external circumstances

These errors can be caught and handled by your program

Runtime Exceptions

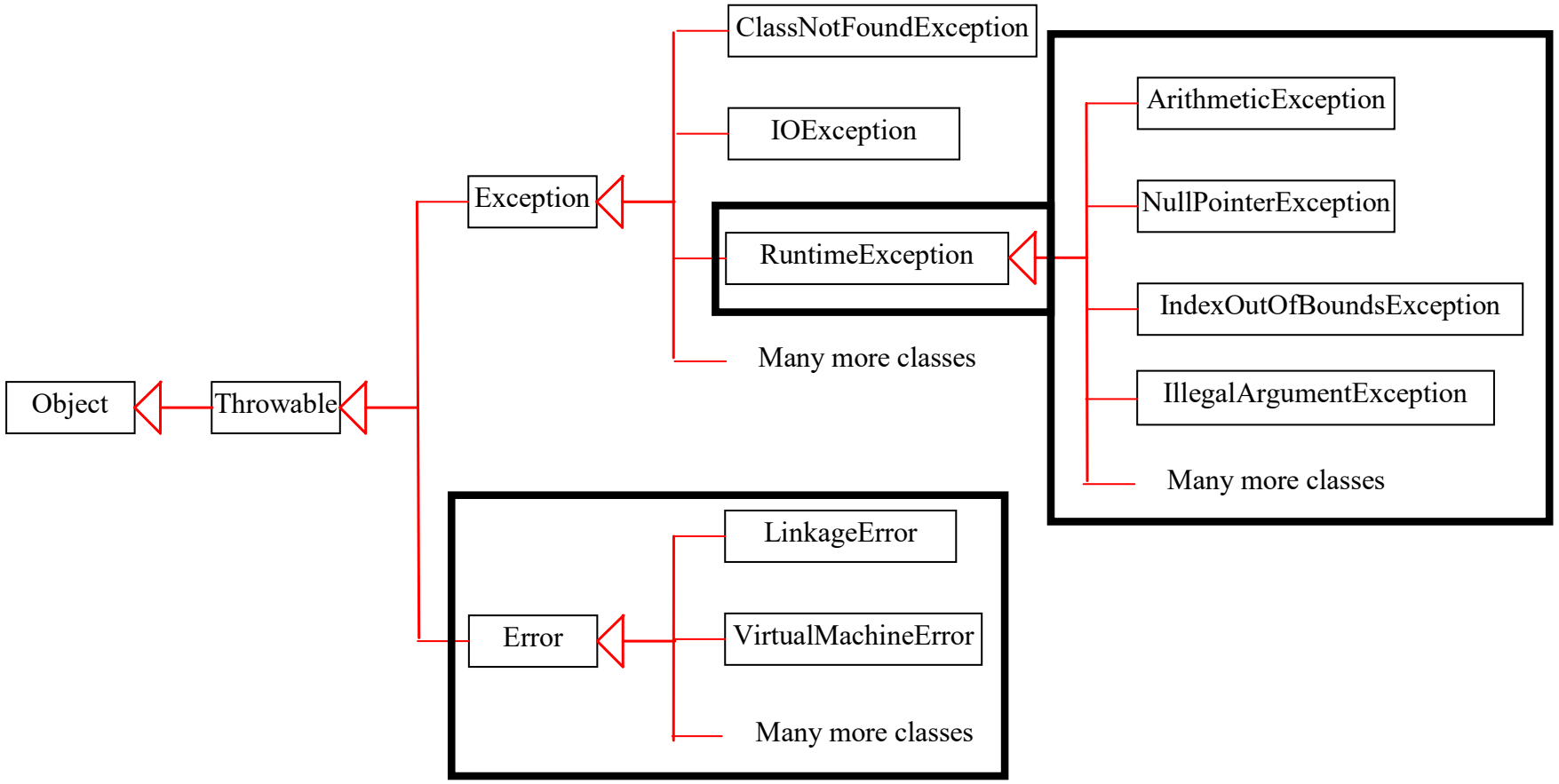


Runtime Exceptions are caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors

Checked Exceptions versus Unchecked Exceptions

- RuntimeException + Error & their subclasses are known as **unchecked exceptions**
- All other exceptions are known as checked exceptions:
  - The compiler forces the programmer to check and deal with the exceptions

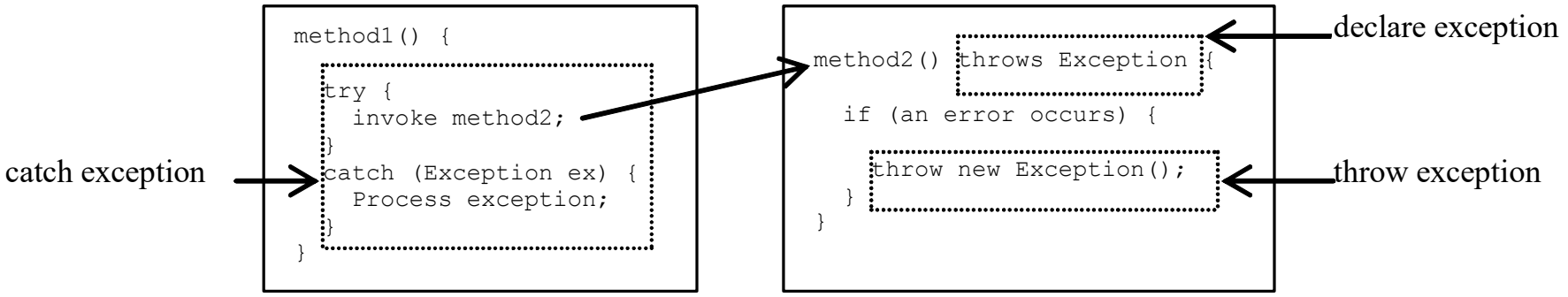
Unchecked Exceptions



Unchecked Exceptions

- In most cases, unchecked exceptions reflect programming logic errors that are not recoverable
  - *NullPointerException*: access an object through a reference variable before an object is assigned to it
  - *IndexOutOfBoundsException*: access an element in an array outside the bounds of the array
- These are the logic errors that should be corrected in the program
- Unchecked exceptions can occur anywhere in the program
- To avoid cumbersome overuse of try-catch blocks, Java **does not mandate** to write **code** to **catch unchecked exceptions**

Declaring, Throwing, and Catching Exceptions



- Every method must state the types of checked exceptions it might throw = declaring exceptions
- `public void myMethod() throws IOException`
- `public void myMethod() throws IOException, OtherException`

Throwing Exceptions

- When the program detects an error, the program can create an instance of an appropriate exception type and throw it = throwing an exception

```
throw new TheException();
```

```
TheException ex = new TheException();
throw ex;
```

Throwing Exceptions Example

```
/** Set a new radius */
public void setRadius(double newRadius)
     throws IllegalArgumentException {
   if (newRadius >= 0)
     radius = newRadius;
   else
     throw new IllegalArgumentException(
       "Radius cannot be negative");
}
```

Catching Exceptions

```
try {
  statements;//Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  //handler for exception1;
}
catch (Exception2 exVar2) {
  //handler for exception2;
}
...
catch (ExceptionN exVar3) {
  //handler for exceptionN;
}
```

Catching Exceptions

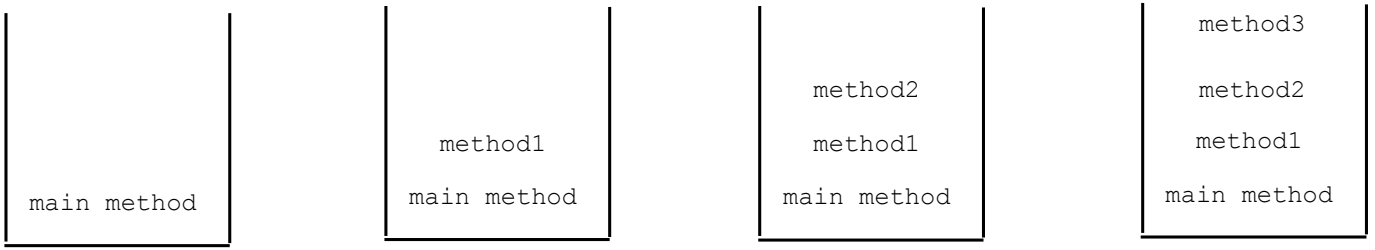
```
main method {
  ...
  try {
    ...
    invoke method1;
    statement1;
  }
  catch (Exception1 ex1) {
    Process ex1;
  }
  statement2;
}
```

```
method1 {
  ...
  try {
    ...
    invoke method2;
    statement3;
  }
  catch (Exception2 ex2) {
    Process ex2;
  }
  statement4;
}
```

```
method2 {
  ...
  try {
    ...
    invoke method3;
    statement5;
  }
  catch (Exception3 ex3) {
    Process ex3;
  }
  statement6;
}
```

An exception is thrown in method3

Call Stack



Catching Exceptions

- You can keep throwing an exception down through multiple methods on the stack
- When you get to the main() method, you can throw the exception out of main(), as well => JVM halting and the stack trace will be printed to the output

```
class TestEx {
  public static void main (String [] args) {
    doStuff();
  }
  static void doStuff() {
    doMoreStuff();
  }
  static void doMoreStuff() {
    int x = 5/0; // Can't divide by zero!
    // throw ArithmeticException
  }
}
```

Catch or Declare Checked Exceptions

- Java forces you to deal with checked exceptions
- If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

Rethrowing Exceptions

```
try {
  statements;
}
catch(TheException ex) {
  //perform operations before exits;
  throw ex;
}
```

The `finally` Clause

```
try {
  statements;
}
catch(TheException ex) {
  //handling ex;
}
finally {
  //finalStatements;
}
```

- Just because finally is invoked does not mean it will complete
- Code in the finally block could itself raise an exception or issue a System.exit().

Constraints

- It is illegal to use a **try clause without either** a *catch* clause or a *finally* clause
- A try clause by itself will result in a compiler error
- Any catch clauses must immediately follow the try block
- Any finally clause must immediately follow the **last** catch clause (or it must immediately follow the try block, if there is *no* catch)
- If a method does not deal with a certain checked exception, it must throw it!

Cautions When Using Exceptions

- Exception handling *separates error-handling code from normal programming tasks*, thus making programs easier to read and to modify
- Be aware that exception handling usually requires more time and resources because
  - It requires instantiating a new exception object
  - Rolling back the call stack
  - Propagating the errors to the calling methods

Exception Matching (1)

- When an exception is thrown, Java will try to find (by looking at the available catch clauses from the top down) a catch clause for the exception type
- If it does not find one, it will search for a handler for a supertype of the exception
- If it does not find a catch clause that matches a supertype for the exception, then the exception is propagated down the call stack

Exception Matching (2)

- The handlers for the **most specific exceptions** must always be **placed above** those for more general exceptions

```
try {
// do risky IO things
} catch (IOException e) {
// handle general IOExceptions
} catch (FileNotFoundException ex) {
// handle just FileNotFoundException
}
```

- If you have a catch clause for both IOException and Exception, you must put the catch for IOException first in your code
- Otherwise, the IOException would be caught by catch(Exception e) because a catch argument can catch the specified exception or any of its subtypes
- *The compiler will stop you from defining catch clauses that can never be reached*

When to Throw Exceptions

- An exception occurs in a method
- If you want the exception to be processed by its caller, you should create an exception object and throw it
- If you can handle the exception in the method where it occurs, there is no need to throw it

When to Use Exceptions

- Use it to deal with unexpected error conditions
- Do **not** use it to deal with simple, expected situations

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```

When to Use Exceptions

Checking for null objects

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

Bad Exception Handling

```
class TestEx {
  public static void main (String [] args) {
    badMethod();
  }
  static void badMethod() { // No need to declare an Error
    doStuff();
  }
  static void doStuff() { //No need to declare an Error
    try {
      throw new Error();
    }
    catch(Error me) {
      throw me; // We catch it, but then rethrow it
    }
  }
}
```

Multi-catch Exceptions

- Eliminate duplicate code
- Note: If a catch block handles more than one exception type, then the catch parameter is **implicitly final**
  - e.g., the catch parameter *ex* is final and you cannot assign any values to it within the catch block

```
catch (IOException ex) {
  logger.log(ex);
  throw ex;
}
catch (SQLException ex) {
  logger.log(ex);
  throw ex;
}
```

```
catch (IOException |
SQLException ex) {
  logger.log(ex);
  throw ex;
}
```

Rethrowing Exceptions with More Inclusive Type Checking

- Java SE 7+ compiler performs more precise analyses of rethrown exceptions
  - Benefit: specify more specific exception types in the throws clause of a method declaration

```
class FirstException extends Exception { }
class SecondException extends Exception { }
public void rethrowException (String exceptionName)
 throws Exception {
    try {
      if (exceptionName.equals("First")) {
        throw new FirstException();
      } else {
        throw new SecondException();
      }
    } catch (Exception e) {
      throw e;
    }
  }
```

Rethrowing Exceptions with More Inclusive Type Checking

```
public void rethrowException(String exceptionName)
 throws FirstException, SecondException {
    try {
      // ...
    }
    catch (Exception e) {
      throw e;
    }
  }
```

- The compiler verifies that the type of the rethrown exception meets the following conditions:
  - The try block can throw it
  - There are no other preceding catch blocks that can handle it
  - It is a subtype or supertype of one of the catch clause's exception parameters

Defining Custom Exception Classes

- Use the exception classes in the Java JRE/JDK API whenever possible
- Define custom exception classes if the predefined classes are not enough
- Define custom exception classes by extending Exception or a subclass of Exception
- Your exception is considered a *checked exception* and the compiler will enforce the handle or declare rule for that exception

Custom Exception Class Example

```
class MyException extends Exception {
  void someMethod () {
    doStuff();
  }
  void doStuff() throws MyException {
    try {
      throw new MyException();
    }
    catch(MyException me) {
      throw me;
    }
  }
}
```

Exception handling with method overriding in Java

- An overriding method **can throw any unchecked exceptions**, regardless of whether the overridden method throws exceptions or not
- The overriding method **must not throw checked exceptions** that are **new** or **broader** than the ones declared by the overridden method
- The overriding method can **throw narrower or fewer exceptions** than the overridden method

Overwriting Methods

SuperClass	SubClass
Does not declare an exception	Can only declare unchecked exceptions, but not checked exceptions
Declares an exception	Can only declare child exception(**s**) of the exception declared by the super class, but **no new** checked exceptions
	Can be declared without exceptions

Best practices to catch, print, and re-throw Java exceptions

- **Do not ignore** Exceptions
  - e.g., never suppress caught exceptions with blank catch blocks!
- Never consume the exception in a catch block (i.e., return null)
- Prefer specific `exceptions`
  - Declare the particular checked exceptions which can be thrown by your method, not “throws Exception()”
  - Don’t catch Throwable
- Never throw an exception from the finally block
- Do not Log and Throw
  - Generates multiple error messages for the same exception ☹

Noooooo

```
// Worst - no indication that an exception occurred and processing
continues
try {
     // do work
} catch (Throwable t) {
}

// Very Bad - an exception occurred, but there is no stack trace &
processing continues
try {
     // do work
} catch (Throwable t) {
     System.err.println("Error: " + t.getMessage());
}

// Incorrect - The stack trace of the original exception is lost (e.g.,
NullPointerException is thrown, getMessage() returns a blank string =>
little indication of the problem)
try {
     // do work
} catch (Throwable t) {
     throw new IOException("Error: " + t.getMessage());
}
```

Assertions

Assertions

- A Java statement that enables you to assert an assumption about your program
- Contains a Boolean expression that should be true during program execution
- Can be used to assure program correctness and avoid logic errors

Declaring Assertions

- An assertion is declared using the new Java keyword assert in JDK 1.4+ as follows:
  - assert assertion; or
  - assert assertion : detailMessage;
- where assertion is a Boolean expression and detailMessage is a primitive-type or an Object value.

Assertion Examples

```
void noReturn() { }
int aReturn() { return 1; }
void go() {
  int x = 1;
  boolean b = true;
  // legal assert statements
  assert(x == 1);
  assert(b);
  assert true;
  assert(x == 1) : x;
  assert(x == 1) : aReturn();
  assert(x == 1) : new ValidAssert();
  // ILLEGAL assert statements
  assert(x = 1); // none of these are booleans
  assert(x);
  assert 0;
  assert(x == 1) : ; // none of these return a value
  assert(x == 1) : noReturn();
  assert(x == 1) : ValidAssert va;
}
```

Executing Assertions

- When an assertion statement is executed, Java evaluates the assertion
  - If **false**, an AssertionError will be thrown
- The AssertionError class has a no-arg constructor and *seven overloaded* single-argument constructors: int, long, float, double, boolean, char, and Object
- Since AssertionError is a subclass of **Error**, when an assertion becomes false, the program displays a message on the console and exits

Executing Assertions Example

```
public class AssertionDemo {
  public static void main(String[] args) {
    int i, sum = 0;
    for (i = 0; i < 10; i++) {
      sum += i;
    }
    assert i == 10;
    assert sum > 10 && sum < 5 * 10:"sum:" + sum;
  }
}
```

Running Programs with Assertions

- By default, assertions are disabled at runtime
- To enable them, use the switch -enableassertions, or -ea for short, as follows:
  - java -ea AssertionDemo
- Assertions can be selectively enabled or disabled at class level or package level
- Disable switch: -disableassertions or -da for short:
  - java -ea:package1 -da:Class1 AssertionDemo

Assertions in Intellij

- Run → Edit Configurations
- Select *Modify options* & choose *Add VM options*

Appropriate usage

(versus legal declarations)

Using Exception Handling or Assertions (1)

- Assertion should not be used to replace exception handling
- Exception handling deals with unusual circumstances during program execution
- **Assertions** are to ensure the **correctness** of the program
- **Exception handling** addresses **robustness**
- Like exception handling, assertions are not used for normal tests, but for internal consistency and validity checks
- Assertions are checked at runtime and can be turned on or off at startup time

Using Exception Handling or Assertions (2)

- Do use assertions to validate arguments to a private methods
- Do **not** use assertions for argument checking in public methods (inappropriate)
- Valid arguments that may be passed to a public method are a part of the method's contract
- The contract must always be obeyed whether assertions are enabled or disabled

```
public void setRadius(double newRadius) {
  assert newRadius >= 0;
  radius =  newRadius;
}
```

Using Exception Handling or Assertions (3)

- Do **not** use assertions to validate command-line arguments
- Use assertions to reaffirm assumptions
- This gives you more confidence to ensure correctness of the program
- A common use of assertions is to replace assumptions with assertions in the code

Using Exception Handling or Assertions (4)

- Use assertions, even in public methods, to check for cases that you know are never ever supposed to happen

```
switch (month) {
  case 1: ... ; break;
  case 2: ... ; break;
  ...
  case 12: ... ; break;
  default: assert false : "Invalid month: " + month;
}
```

Using Exception Handling or Assertions (5)

- Use `assert false;` for code that should never be reached
  - An assertion error is thrown immediately if the assert statement is executed

Using Exception Handling or Assertions (6)

- Do **not** use assert expressions that can cause side effects
- Assertions are not guaranteed to always run
  => you do not want behavior that changes depending on whether assertions are enabled or not

```
public void doStuff() {
  assert (modifyThings());
  // continues on
}
public boolean modifyThings() {
  y = x++;
  return true;
}
```